# Rebuilding and checking PDF documentation

Markdown is the source of truth. The adjacent PDF is a generated deliverable; never edit it independently. This process builds documentation only. It does not install or configure lab infrastructure.

## 1. Toolchain

The build uses free Python libraries pinned in `requirements-docs.txt`: ReportLab for PDF layout/vector drawing, Mistune for Markdown parsing, Pygments for code highlighting, and pypdf for structural/text/link validation. Pillow and charset-normalizer are pinned ReportLab dependencies. No paid application, office suite, browser automation, or remote rendering service is required.

The task environment provides Python 3.12.14, ReportLab 4.4.9, pypdf 6.10.0, and Poppler 26.07.0. Mistune 3.4.0 and Pygments 2.21.0 were added only to an ignored local dependency directory. A normal Windows checkout can use its own documentation virtual environment instead of depending on Codex paths.

## 2. Windows setup and one-command rebuild

Run from the repository root with an existing Python 3.11 or later installation. Create a local documentation environment and install the pinned free dependencies when needed:

```
python -m venv .venv-docs
.\.venv-docs\Scripts\python.exe -m pip install -r requirements-docs.txt
```

Rebuild all PDF editions with one command:

```
.\.venv-docs\Scripts\python.exe scripts/build_docs.py
```

With the environment activated, the equivalent is `python scripts/build_docs.py`. The build works on Windows and Linux. Installation is limited to documentation dependencies in the chosen environment; it has no Proxmox, VM, firewall, network, directory, or production operation.

## 3. Generated files and consistency

Every maintained user-facing Markdown document has a PDF beside it. The approved source set includes root guides, purpose-specific documents, task reports/addenda, folder instructions, and public evidence guidance/records. The two decision/risk compatibility redirects are excluded as duplicate navigation pages. Unknown source/PDF pairs fail validation.

`docs/documentation-manifest.json` records relative source/PDF paths, SHA-256 hashes, page count, source text-token coverage, links/bookmarks, and the toolchain/renderer fingerprint. The document version shown in each PDF is derived from its source hash, not the current date or Git HEAD. Fixed PDF metadata and embedded package fonts make output reproducible. Unchanged inputs skip generation, and byte-identical output is not rewritten.

The build preserves paragraph/list/table content, code, and links. It numbers PDF sections, repeats table headers across pages, wraps long cells/code, creates page numbers and navigation, and gives longer documents a title page and contents. It renders this repository's Mermaid flowchart subset as a vector figure with the same nodes/edges/labels; an unsupported diagram fails rather than disappearing. Syntax highlighting changes presentation, not source commands. Typography converts Unicode dashes and unsupported arrow glyphs to readable ASCII equivalents; Markdown remains unchanged.

External URLs are retained. Same-document anchors become PDF navigation. Cross-document links point to the corresponding public PDF edition; historical decision/risk links resolve through the documented

aliases. PDF reader support for external/named destinations varies. Code blocks are reference text, not active commands or executable attachments.

## 4. Validation and visual review

Run these commands after a rebuild:

```
.\.venv-docs\Scripts\python.exe scripts/build_docs.py --check --rebuild-check
python scripts/validate_docs.py --require-finalized
.\.venv-docs\Scripts\python.exe -m unittest discover -s tests -v
git diff --check
```

The PDF check opens every output, checks text extraction/searchability, page/footer/navigation/link structure, coverage of source text with layout line-break normalization, and source/PDF hashes. The rebuild check renders fresh PDFs in memory and compares bytes. Documentation validation checks migrated links, requirement/stage/report structure, publication patterns, and finalized-history preservation. CI installs the pinned libraries, runs these checks, and never connects to the lab.

Text extraction does not establish visual quality. Render representative pages using an available Poppler installation and inspect the result:

```
pdftoppm -f 1 -l 3 -r 110 -png docs/assignments/PROJECT-ASSIGNMENT.pdf tmp/pdfs/assignment
```

Review title/contents pages, long tables, the architecture figure, command blocks, historical report appendices, and page transitions. Check for clipping, overlap, missing glyphs, unreadable type, broken rows, and incorrect headers/footers. Keep previews under ignored `tmp/pdfs/`; do not publish raw screenshots or local environment details. If Poppler is absent, obtain an approved local renderer before claiming visual review. No renderer is assumed installed by this guide.

## 5. Troubleshooting and recovery

- Missing library: use the documentation environment and pinned requirements; do not modify global software or install lab products.
- Stale/missing PDF or manifest entry: rebuild from source, then rerun both checks. Do not edit a PDF or hash entry to hide a mismatch.
- Layout failure: fix the renderer or editable source, rebuild, and inspect affected pages again. Keep finalized historical report Markdown unchanged.
- Broken link after a move: update editable references and the source/output map. Retain compatibility navigation when an immutable historical source needs it.
- Unexpected binary change: compare source, renderer, font, and toolchain fingerprints. A no-change rebuild must leave bytes unchanged.
- Failed publication: retain the reviewed branch, inspect actual CI errors, and reconcile through normal Git; never force-push away work or rewrite finalized reports.

The project license remains an owner decision. This toolchain uses existing free dependencies and does not select a software license for the repository.